

claude-windows / c7679509-ee2b-4443-b240-0b3b1b1a7291

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c7679509-ee2b-4443-b240-0b3b1b1a7291\subagents\workflows\wf_a2b97cff-dcd\agent-a3853e3d0009ddd5f.jsonl`  
- SHA-256: `70ef06df6317d5003a83e2c5787f1a9edebc63f7446429d056753223d26c40eb`  
- Source modified: `2026-06-21T14:05:18+00:00`  
- Imported at: `2026-07-05T16:48:23+00:00`  
- project: `wf_a2b97cff-dcd`  
- session_id: `c7679509-ee2b-4443-b240-0b3b1b1a7291`
```

Transcript

1. user (2026-06-21T14:01:52.121Z)

Adversarially VERIFY the load-bearing claims below about packaging/distributing our Python app. For EACH claim, try to REFUTE it (check feasibility, licensing accuracy, version facts, whether it actually applies to a torch+opencv+ffmpeg+FastAPI app). Default to "uncertain" if you cannot confirm. Use web search to check current facts.

TOPIC: weights-and-ffmpeg-bundling

CLAIMS:

```
[  
  {  
    "claim": "PyPI default limits a single uploaded file to 100 MiB and a whole project to 10 GiB; larger files require a per-project limit-increase request after a sub-limit release exists. This caps the vendor-weights-in-wheel option.",  
    "confidence": "high",  
    "source": "https://docs.pypi.org/project-management/storage-limits/"  
  },  
  {  
    "claim": "imageio-ffmpeg bundled ffmpeg binary is effectively a GPL build because it uses libx264 as the default mp4 codec and libx264 requires enable-gpl; redistributing it in our installer would impose GPL on the bundle.",  
    "confidence": "high",  
    "source": "https://github.com/imageio/imageio-ffmpeg + https://www.ffmpeg.org/legal.html"  
  },  
  {  
    "claim": "imageio-ffmpeg bundles only ffmpeg, not ffprobe. The engine hard-requires ffprobe for video duration at backend/utils/video.py line 11 and compiler.py line 113, so imageio-ffmpeg alone cannot satisfy the engine.",  
    "confidence": "high",  
    "source": "https://github.com/imageio/imageio-ffmpeg/issues/23"  
  },  
  {  
    "claim": "The imageio-ffmpeg wrapper is BSD-2-Clause; binary resolution order is the IMAGEIO_FFMPEG_EXE env var, then the bundled binary, then conda, then system PATH, and the env var is used directly without validation.",  
    "confidence": "high",  
    "source": "https://github.com/imageio/imageio-ffmpeg/blob/main/imageio_ffmpeg/_utils.py"  
  },  
  {  
    "claim": "gyan.dev and BtbN GPL static Windows builds installed by winget Gyan.FFmpeg or BtbN.FFmpeg.GPL are GPLv3 and include libx264, libx265, and ffprobe. Fine to use when user-installed but not to redistribute in an MIT, Apache, BSD-only bundle.",  
    "confidence": "high",  
    "source": "https://www.gyan.dev/ffmpeg/builds/ + https://github.com/BtbN/FFmpeg-Builds/issues/29"  
  },  
  {  

```

```

    "claim": "BtbN LGPL builds omit GPL components and so exclude libx264; H.264 in an LGPL
build needs the native LGPL encoder or libopenh264 which is BSD. Shipping an LGPL ffmpeg
therefore requires changing the engine c:v libx264 default.",
    "confidence": "high",
    "source": "https://github.com/BtbN/FFmpeg-Builds + https://www.ffmpeg.org/legal.html"
  },
  {
    "claim": "Embedding ffmpeg built with enable-gpl makes the whole build GPL and forbids
embedding it in a non-GPL redistributed bundle; LGPL builds allow redistribution subject to re-
link or source-offer obligations. The clean escape is to not redistribute ffmpeg and have the
user system-install it.",
    "confidence": "high",
    "source": "https://www.ffmpeg.org/legal.html"
  },
  {
    "claim": "huggingface_hub is Apache-2.0 and provides hf_hub_download with a content-hashed
cache, resumable downloads, and an offline switch via HF_HUB_OFFLINE or local_files_only, giving
a batteries-included first-run downloader.",
    "confidence": "high",
    "source": "https://pypi.org/project/huggingface-hub/ +
https://huggingface.co/docs/huggingface_hub/en/package_reference/file_download"
  },
  {
    "claim": "WASB-SBDT from nttcom is MIT-licensed, so its pretrained shuttle weights are
redistributable and we can self-host them on our GCS bucket or on HF. Verify each weight file
actual host and provenance before mirroring, since the README points at MODEL_ZOO.md and some
academic weights are Google-Drive-hosted.",
    "confidence": "medium",
    "source": "https://github.com/nttcom/WASB-SBDT"
  },
  {
    "claim": "The engine already has the seams for download-on-first-run:
GCSStorage.fetch_to_local is overwrite-aware and does mkdir plus download, and a WASB_WEIGHTS
env override is consumed by NativeWasbConfig.from_indexing_cfg with env winning over config, so
weight-fetch can set WASB_WEIGHTS with no detector code change.",
    "confidence": "high",
    "source": "backend/storage/gcs.py line 30 ; backend/pipeline/detectors/native_wasb_runner.py
lines 80-97"
  },
  {
    "claim": "ffmpeg and ffprobe are invoked as bare PATH strings via subprocess:
backend/utls/video.py uses ffprobe at line 11 and ffmpeg at line 33; the stitcher uses ffprobe
at compiler.py line 113, c:v libx264 at line 253, and drawtext or overlay watermark at lines
179-185. A single resolver helper is the minimal change for a configurable or bundled binary.",
    "confidence": "high",
    "source": "backend/utls/video.py ; backend/pipeline/stitcher/compiler.py"
  },
  {
    "claim": "scripts/doctor.py already does first-run checks: shutil.which for ffmpeg and
ffprobe, GPU detection, torch out-of-band index selection cu124 or cpu, and on Windows it prints
exactly the winget install command for Gyan.FFmpeg. The recommended first-run setup extends this
rather than adding new tooling.",
    "confidence": "high",
    "source": "scripts/doctor.py lines 84-108 and 15-34"
  },
  {
    "claim": "PyInstaller-freezing with CUDA torch yields about 2.6 to 4 GB bundles because CUDA
and cuDNN libs cannot be split; small bundles require CPU-only torch. This argues against a
frozen all-in-one binary and for pip or installer plus on-demand weights plus system ffmpeg.",
    "confidence": "high",
    "source": "https://github.com/orgs/pyinstaller/discussions/8552"
  }
]

```

RECOMMENDATION UNDER TEST: Split the two concerns and reuse the existing seams. WEIGHTS:
download-on-first-run from a public read-only GCS bucket we control (WEIGHTS-A), with Hugging

Face Hub (WEIGHTS-B) as a strong lower-ops alternative. Do not vendor weights in the wheel because the PyPI 100 MiB per-file cap, the bloat for Gemini-only users, and the coupling of model cadence to release cadence all argue against it. Mechanics: a small fetcher resolves WASB_WEIGHTS by precedence of env var, then config, then a per-user cache directory; it downloads over HTTPS where a public bucket needs no google-cloud-storage dependency or credentials; it verifies a pinned SHA-256; it caches once for offline reruns; and it shows a loud resumable progress bar rather than a silent hang. This reuses GCSStorage.fetch_to_local and the WASB_WEIGHTS env seam so the detector code is untouched. Weights are license-clean because WASB-SBDT is MIT, but verify each weight file actual host and provenance before mirroring. FFMPEG: do not redistribute ffmpeg, since that is exactly where the GPL versus LGPL problem bites. Instead route every ffmpeg and ffprobe call through one resolver helper that checks a configured path or the FFMPEG_BINARY env var, then shutil.which, then a known install dir, and otherwise fails loud with the exact winget command; and have first-run setup, extending scripts/doctor.py, detect a missing binary and offer or auto-run the winget install of Gyan.FFmpeg. The full GPL build ships ffprobe, libx264, and libfreetype, so the engine stitch, watermark, and duration probe all work unchanged, and because the user installs ffmpeg we never redistribute GPL bits and stay MIT-clean. Reject imageio-ffmpeg as the primary path because it lacks ffprobe, which the engine requires, and its binary is effectively a GPL build, so it neither fully works nor is redistribution-clean. Keep the LGPL-bundle option FFMPEG-C, a Btbn LGPL build with the encoder swapped from libx264 to libopenh264 or NVENC, as a documented fallback only if a future truly zero-touch installer must avoid the winget step, noting it forces an encoder change and LGPL re-link or source-offer compliance. Net first-run for a non-tech Windows user: the installer or pip drops the app; doctor-style setup checks ffmpeg and ffprobe and, if absent, runs winget with consent and progress; and on the first trajectory run the weight auto-downloads, checksummed and cached. Gemini-only users never trigger a weights download and, if they already have ffmpeg, need nothing else.

2. assistant (2026-06-21T14:01:55.974Z)

I'll verify each claim adversarially. Let me start by examining the actual codebase files referenced in the claims, then web-search to check the external facts.

3. user (2026-06-21T14:01:57.256Z)

```

1      import subprocess
2      import logging
3      from typing import Optional
4
5      logger = logging.getLogger(__name__)
6
7      def get_video_duration(video_path: str) -> float:
8          """Returns the exact duration of a video file in seconds using ffprobe.
9          Returns 0.0 on failure."""
10         try:
11             duration_output = subprocess.check_output([
12                 "ffprobe", "-v", "error", "-show_entries", "format=duration",
13                 "-of", "default=noprint_wrappers=1:nokey=1", video_path
14             ]).decode().strip()
15             return float(duration_output)
16         except Exception as e:
17             logger.error(f"Could not detect video duration for {video_path}: {e}")
18             return 0.0
19
20     def extract_video_chunk(video_path: str, start_time: float, duration: float,
21                             output_path: str, reencode: bool = False,
22                             scale_width: Optional[int] = None) -> bool:
23         """Extracts a chunk from a video file.
24
25         Args:
26             reencode: If True, re-encodes with libx264 (slower but frame-accurate).
27                       If False, uses stream-copy (fast but cuts at nearest keyframe).
28                       Use True for short clips where keyframe drift matters.
29             scale_width: If set (requires reencode), downscale to this width (height auto,
30                          even). Useful for high-res sources headed to an API that analyses at
31                          low res anyway ? shrinks uploads with no analysis loss.
```

```

32     """
33     cmd = [
34         "ffmpeg", "-y",
35         "-ss", str(start_time),
36         "-t", str(duration),
37         "-i", video_path,
38     ]
39     if reencode:
40         if scale_width:
41             cmd += ["-vf", f"scale={int(scale_width)}:-2"]
42             cmd += ["-c:v", "libx264", "-preset", "ultrafast", "-crf", "23", "-c:a", "aac"]
43         else:
44             cmd += ["-c", "copy"]
45     cmd.append(output_path)
46     try:
47         subprocess.run(cmd, check=True, stdout=subprocess.DEVNULL,
stderr=subprocess.DEVNULL)
48         return True
49     except Exception as e:
50         logger.error(f"Failed to extract chunk {start_time}s - {start_time+duration}s:
{e}")
51         return False
52

```

4. user (2026-06-21T14:01:57.640Z)

```

1     import sys
2     import os
3     import subprocess
4     import shutil
5
6     # This diagnostics/bootstrap script lives in scripts/ but operates on the repo
7     # root (requirements.txt, config.json). Resolve the root from this file's
8     # location so it works regardless of the caller's cwd.
9     REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
10
11     def check_command(cmd):
12         """Check if a system command is available."""
13         return shutil.which(cmd) is not None
14
15     def run_pip_install():
16         """Install required Python packages."""
17         print("Installing Python packages from requirements.txt...")
18
19         # Check if NVIDIA GPU is present
20         has_gpu = check_command("nvidia-smi")
21
22         requirements_path = os.path.join(REPO_ROOT, "requirements.txt")
23         pip_cmd = [sys.executable, "-m", "pip", "install", "--user", "-r",
requirements_path]
24
25         import platform
26         if has_gpu:
27             print("[INFO] NVIDIA GPU detected. Configuring PyTorch to download CUDA-enabled
wheels.")
28             pip_cmd.extend(["--extra-index-url", "https://download.pytorch.org/whl/cu124"])
29         elif platform.system() == "Darwin":
30             # macOS has no CUDA; the default PyPI wheel already ships CPU + Apple MPS.
31             print("[INFO] macOS detected. Using default PyTorch wheels (CPU + Apple MPS).")
32         else:
33             print("[INFO] No NVIDIA GPU detected. Defaulting to PyTorch CPU wheels.")
34             pip_cmd.extend(["--extra-index-url", "https://download.pytorch.org/whl/cpu"])
35
36         try:
37             subprocess.check_call(pip_cmd)

```

```

38         print("[SUCCESS] Python packages installed.")
39         return True
40     except subprocess.CalledProcessError as e:
41         print(f"[ERROR] Failed to install Python packages: {e}")
42         return False
43
44     def init_default_config():
45         """Initialize a default config.json if it doesn't exist.
46
47         Now delegates to the canonical backend.config loader so that defaults
48         stay in sync with the Pydantic models (single source of truth).
49         """
50         try:
51             from backend.config import save_default_config
52         except Exception as e:
53             print(f"[ERROR] Could not import backend.config loader: {e}")
54             print("[INFO] Falling back to no-op for config initialization.")
55             return
56
57         config_path = os.path.join(REPO_ROOT, "config.json")
58         if os.path.exists(config_path):
59             print(f"[INFO] {config_path} already exists. Skipping initialization.")
60             return
61
62         try:
63             saved_path = save_default_config(config_path)
64             print(f"[SUCCESS] config.json initialized with default settings at {saved_path}.")
65         except Exception as e:
66             print(f"[ERROR] Failed to create config.json via loader: {e}")
67
68     def run_diagnostics():
69         """Perform system-wide checks for FFmpeg and PyTorch/CUDA."""
70         print("-" * 50)
71         print("SPORTS HIGHLIGHT INDEXER SETUP DIAGNOSTICS")
72         print("-" * 50)
73
74         # 1. Check Python Version
75         print(f"Python Version: {sys.version_info.major}.{sys.version_info.minor}.{sys.version_info.micro}")
76         if sys.version_info.major < 3 or (sys.version_info.major == 3 and sys.version_info.minor < 8):
77             print("[WARNING] Python 3.8+ is highly recommended!")
78         else:
79             print("[PASS] Python version check passed.")
80
81         print("-" * 50)
82
83         # 2. Check FFmpeg / FFprobe
84         ffmpeg_ok = check_command("ffmpeg")
85         ffprobe_ok = check_command("ffprobe")
86
87         if ffmpeg_ok:
88             print("[PASS] FFmpeg is installed and available in PATH.")
89         else:
90             print("[FAIL] FFmpeg is NOT found in your PATH.")
91
92         if ffprobe_ok:
93             print("[PASS] FFprobe is installed and available in PATH.")
94         else:
95             print("[FAIL] FFprobe is NOT found in your PATH.")
96
97         if not (ffmpeg_ok and ffprobe_ok):
98             import platform
99             print("\n" + "!" * 50)

```

```

100         print("CRITICAL: FFmpeg and FFprobe are REQUIRED to run this application.")
101         _sys = platform.system()
102         if _sys == "Windows":
103             print("Install on Windows: winget install Gyan.FFmpeg")
104         elif _sys == "Darwin":
105             print("Install on macOS: brew install ffmpeg")
106         else:
107             print("Install on Linux: sudo apt-get install -y ffmpeg (or your
108 distro's package manager)")
109             print("Or download a build from https://ffmpeg.org/download.html and put
110 ffmpeg/ffprobe on PATH.")
111             print("!" * 50 + "\n")
112         else:
113             print("[SUCCESS] All essential media command line tools are ready.")
114
115     print("-" * 50)
116
117     # 3. Check ML / GPU (Optional but good)
118     try:
119         import torch
120         cuda_ok = torch.cuda.is_available()
121         if cuda_ok:
122             print(f"[PASS] PyTorch loaded successfully. CUDA (GPU) is AVAILABLE:
123 {torch.cuda.get_device_name(0)}")
124         else:
125             print("[INFO] PyTorch loaded successfully, but CUDA (GPU) is NOT available.
126 Falling back to CPU.")
127     except ImportError:
128         print("[INFO] PyTorch is not installed. Relevance checks will run using
129 lightweight visual heuristics.")
130
131     print("=" * 50)
132     return ffmpeg_ok and ffprobe_ok
133
134 if __name__ == "__main__":
135     init_default_config()
136     media_ok = run_diagnostics()
137     run_pip_install()
138     print("\n[INFO] Setup complete. If diagnostics passed, you can now run the app!")
139

```

5. assistant (2026-06-21T14:02:00.280Z)

Let me read the remaining referenced files in the codebase.

6. user (2026-06-21T14:02:01.631Z)

```

1     import os
2     import subprocess
3     import tempfile
4     import logging
5     from pathlib import Path
6     from typing import Any, Dict, List, Optional, Tuple, Union
7
8     logger = logging.getLogger(__name__)
9
10    # Bundled, commercially-clean default watermark font (Apache-2.0). Shipped so the
11    overlay
12    # renders out-of-the-box without a system fontconfig setup (a common gap on Windows,
13    where
14    # `font='Sans` fails). Overridable via stitching.watermark.font_path. This module lives
15    at
16    # backend/pipeline/stitcher/, so parents[2] == backend/.
17    _BUNDLED_FONT = Path(__file__).resolve().parents[2] / "assets" / "fonts" / "RobotoSlab-
18    Regular.ttf"

```

```

15
16 class VideoCompiler:
17     def __init__(self, output_dir: str, end_padding: float = 1.0, begin_padding: float =
0.5,
18                 watermark: Optional[Any] = None):
19         self.output_dir = output_dir
20         self.end_padding = end_padding
21         self.begin_padding = begin_padding
22         # Optional branding overlay. Accepts a WatermarkConfig model or a plain dict;
23         # normalized to a dict (or None) so this module stays config-package-agnostic.
24         self.watermark = self._normalize_watermark(watermark)
25         os.makedirs(output_dir, exist_ok=True)
26
27     @staticmethod
28     def _normalize_watermark(watermark: Optional[Any]) -> Optional[Dict[str, Any]]:
29         if watermark is None:
30             return None
31         if hasattr(watermark, "model_dump"):
32             return watermark.model_dump()
33         if isinstance(watermark, dict):
34             return dict(watermark)
35         return None
36
37     @staticmethod
38     def _ff_quote(path: str) -> str:
39         """Make a filesystem path safe to embed inside single quotes in an ffmpeg
40         filtergraph operand (drawtext fontfile/textfile, movie= source): forward
41         slashes (Windows-friendly), escaped single quotes, AND escaped colons.
42
43         The colon MUST be escaped even inside single quotes: ffmpeg's filtergraph
44         option parser treats ':' as the option separator, so a Windows drive-letter
45         path like ``C:/dir/font.ttf`` otherwise truncates the operand at ``C`` and
46         the encode fails with "Invalid argument". This silently disabled the
47         on-by-default bundled-font watermark (and absolute logo paths) on Windows."""
48         return path.replace("\\", "/").replace("'", r"\'").replace(":", r"\:")
49
50     @staticmethod
51     def _parse_time(val: Union[int, float, str]) -> float:
52         """Normalizes a timestamp value to float seconds.
53         Handles: float/int (pass-through), plain numeric strings ("12.4"),
54         and colon-separated strings ("MM:SS", "HH:MM:SS", "H:MM:SS.ms").
55         This mirrors the parse_time logic used in gemini.py to ensure
56         the stitcher can always consume whatever format the indexer produced."""
57         if isinstance(val, (int, float)):
58             return float(val)
59         if isinstance(val, str):
60             val = val.strip()
61             if ":" in val:
62                 # Handle MM:SS, HH:MM:SS, etc.
63                 parts = val.split(":")
64                 parts.reverse()
65                 total = 0.0
66                 for i, p in enumerate(parts):
67                     try:
68                         total += float(p) * (60 ** i)
69                     except ValueError:
70                         pass
71                 return total
72             try:
73                 return float(val)
74             except ValueError:
75                 logger.warning(f"Could not parse timestamp value '{val}' as a number.
Defaulting to 0.0.")
76                 return 0.0
77                 logger.warning(f"Unexpected timestamp type {type(val).__name__} for value

```

```

'val}'. Defaulting to 0.0.")
78         return 0.0
79
80     @staticmethod
81     def _merge_overlapping(segments: List[Tuple[float, float]]) -> List[Tuple[float,
float]]:
82         """Normalize, sort, and merge overlapping/duplicate (start, end) ranges.
83
84         A highlight reel must never replay the same footage. The same rally can land in
85         the segment list more than once ? e.g. two detector runs accumulated for one
86         video (add_video is an UPSERT, so re-running without a prune leaves both sets),
87         or
88         chunk-overlap duplicates near boundaries. Stitching those verbatim plays the
89         rally
90         twice. Merging any range that overlaps or abuts the previous one collapses true
91         duplicates/overlaps while preserving genuinely distinct (gapped) rallies.
92         Degenerate rows (end <= start) are dropped. Timestamps may be float/int/str.
93         """
94         parsed = []
95         for raw_s, raw_e in segments:
96             s = VideoCompiler._parse_time(raw_s)
97             e = VideoCompiler._parse_time(raw_e)
98             if e > s:
99                 parsed.append((s, e))
100         parsed.sort()
101         merged: List[Tuple[float, float]] = []
102         for s, e in parsed:
103             if merged and s <= merged[-1][1]: # overlaps/abuts previous -> extend it
104                 merged[-1] = (merged[-1][0], max(merged[-1][1], e))
105             else:
106                 merged.append((s, e))
107         return merged
108
109     def _get_video_duration(self, video_path: str) -> float:
110         """Probes the video file to get its total duration in seconds.
111         Returns 0.0 if it cannot be determined."""
112         try:
113             result = subprocess.run(
114                 [
115                     "ffprobe", "-v", "error",
116                     "-show_entries", "format=duration",
117                     "-of", "default=noprint_wrappers=1:nokey=1",
118                     video_path
119                 ],
120                 capture_output=True, text=True, check=True
121             )
122             return float(result.stdout.strip())
123         except Exception as e:
124             logger.warning(f"Could not determine video duration via ffprobe: {e}")
125             return 0.0
126
127     def _watermark_filter(self, tmp_dir: str) -> Optional[str]:
128         """Build the ffmpeg -vf filtergraph that burns the configured watermark (text
129         and/or logo PNG) into a clip during the per-clip re-encode. Returns None when
130         watermarking is disabled or has nothing to draw. The concat stage stays -c copy
131         because the mark is already baked into every clip identically."""
132         wm = self.watermark
133         if not wm or not wm.get("enabled"):
134             return None
135
136         text = (wm.get("text") or "").strip()
137         logo_path = (wm.get("logo_path") or "").strip()
138         if logo_path and not os.path.exists(logo_path):
139             logger.warning(f"[watermark] logo not found: {logo_path} ? skipping logo
overlay.")

```



```

138         logo_path = ""
139     if not text and not logo_path:
140         return None
141
142     margin = int(wm.get("margin", 24))
143     opacity = float(wm.get("opacity", 0.85))
144     position = str(wm.get("position", "bottom-right"))
145     # drawtext uses lowercase w/h (main) + tw/th (text); overlay uses W/H (main) +
w/h (logo).
146     dt_xy = {
147         "bottom-right": f"x=w-tw-{margin}:y=h-th-{margin}",
148         "bottom-left": f"x={margin}:y=h-th-{margin}",
149         "top-right": f"x=w-tw-{margin}:y={margin}",
150         "top-left": f"x={margin}:y={margin}",
151     }.get(position, f"x=w-tw-{margin}:y=h-th-{margin}")
152     ov_xy = {
153         "bottom-right": f"W-w-{margin}:H-h-{margin}",
154         "bottom-left": f"{margin}:H-h-{margin}",
155         "top-right": f"W-w-{margin}:{margin}",
156         "top-left": f"{margin}:{margin}",
157     }.get(position, f"W-w-{margin}:H-h-{margin}")
158
159     drawtext = ""
160     if text:
161         ratio = float(wm.get("font_size_ratio") or 0.035)
162         divisor = max(1, round(1.0 / ratio))
163         # textfile= sidesteps the notorious drawtext text= escaping for arbitrary
strings.
164         tf_path = os.path.join(tmp_dir, "watermark.txt")
165         with open(tf_path, "w", encoding="utf-8") as fh:
166             fh.write(text)
167         # Font precedence: explicit font_path ? bundled Apache-2.0 default (renders
168         # without fontconfig) ? fontconfig family (last-resort, environment-
dependent).
169         font_path = (wm.get("font_path") or "").strip()
170         if not font_path and _BUNDLED_FONT.exists():
171             font_path = str(_BUNDLED_FONT)
172         if font_path:
173             font_opt = f"fontfile='{self._ff_quote(font_path)}'"
174         else:
175             font_opt = f"font='{wm.get('font', 'Sans')}'"
176         box = ""
177         if wm.get("box", True):
178             box = f":box=1:boxcolor=black@{min(opacity, 0.5):.2f}:boxborderw=10"
179         drawtext = (
180             f"drawtext=textfile='{self._ff_quote(tf_path)}':{font_opt}"
181             f":fontcolor=white@{opacity:.2f}:fontsize=h/{divisor}:{dt_xy}{box}"
182         )
183
184     if logo_path and drawtext:
185         return
186     f"movie='{self._ff_quote(logo_path)}'[wm];[in][wm]overlay={ov_xy},{drawtext}"
187     if logo_path:
188         return f"movie='{self._ff_quote(logo_path)}'[wm];[in][wm]overlay={ov_xy}"
189     return drawtext
190
191 def _trim_one_segment(
192     self,
193     idx: int,
194     raw_start: Union[int, float, str],
195     raw_end: Union[int, float, str],
196     segments: List[Tuple[float, float]],
197     video_duration: float,
198     tmp_dir: str,
199     raw_video_path: str,

```

```

199         watermark_filter: Optional[str],
200     ) -> Tuple[Optional[str], Optional[Tuple[int, float, float, str]]]:
201         """Trim a single segment into its own clip file (extracted verbatim from the
202         per-segment body of compile_highlights). Returns (clip_path, skip_record): on
203         success the clip path and None; on a skip (invalid/out-of-range/ffmpeg failure)
204         None and the (idx, start, end, reason) record to record in skipped_segments."""
205         # Normalize timestamps to float seconds ? handles float, int,
206         # plain numeric strings, and colon-separated formats like "MM:SS"
207         start = self._parse_time(raw_start)
208         end = self._parse_time(raw_end)
209         segment_label = f"Segment #{idx+1}/{len(segments)} [{start:.2f}s - {end:.2f}s]"
210
211         # Validate the segment times
212         if start < 0:
213             logger.warning(f"{segment_label}: start_time is negative ({start:.2f}s).
Clamping to 0.")
214             start = 0.0
215
216         if start >= end:
217             logger.error(f"{segment_label}: start_time ({start:.2f}s) >= end_time
({end:.2f}s). Skipping invalid segment.")
218             return None, (idx, start, end, "start >= end")
219
220         # Check if segment extends beyond video duration
221         if video_duration > 0:
222             if start >= video_duration:
223                 logger.error(f"{segment_label}: start_time ({start:.2f}s) is beyond the
video duration ({video_duration:.2f}s). "
224                             f"This segment cannot be extracted ? the video ran out.
Skipping.")
225                 return None, (idx, start, end, "start beyond video end")
226
227             if end > video_duration:
228                 original_end = end
229                 end = video_duration
230                 logger.warning(f"{segment_label}: end_time ({original_end:.2f}s) exceeds
video duration ({video_duration:.2f}s). "
231                             f"Clamping end_time to {video_duration:.2f}s. The segment
may be truncated.")
232
233             clip_path = os.path.join(tmp_dir, f"clip_{idx}.mp4")
234
235             # Precise sub-second trim using fast re-encoding to preserve stream headers
236             padded_start = max(0.0, start - self.begin_padding)
237             padded_end = end + self.end_padding
238
239             # Clamp padded_end to video duration if known
240             if video_duration > 0 and padded_end > video_duration:
241                 padded_end = video_duration
242                 logger.info(f"{segment_label}: Padded end ({end + self.end_padding:.2f}s)
exceeds video duration. "
243                             f"Clamping to {video_duration:.2f}s (end padding partially
applied).")
244
245             trim_duration = padded_end - padded_start
246             logger.info(f"Trimming {segment_label} -> [{padded_start:.3f}s -
{padded_end:.3f}s] (duration: {trim_duration:.2f}s)")
247
248             base_cmd = [
249                 "ffmpeg", "-y",
250                 "-ss", str(round(padded_start, 3)),
251                 "-to", str(round(padded_end, 3)),
252                 "-i", raw_video_path,
253                 "-c:v", "libx264",
254                 "-preset", "superfast",

```

```

255         "-crf", "22",
256         "-c:a", "aac",
257         "-b:a", "128k",
258     ]
259     vf_args = ["-vf", watermark_filter] if watermark_filter else []
260
261     # Try with the watermark first; if that encode fails (e.g. a bad font/logo
262     # path), retry once WITHOUT it so a branding misconfig can never nuke the
263     # whole reel. Clips stay codec-identical either way, so concat -c copy holds.
264     attempts = [vf_args, []] if vf_args else [[]]
265     produced = False
266     last_exc: Optional[subprocess.CallProcessError] = None
267     cmd = base_cmd + [clip_path]
268     for attempt_vf in attempts:
269         cmd = base_cmd + attempt_vf + [clip_path]
270         try:
271             subprocess.run(cmd, capture_output=True, check=True)
272         except subprocess.CallProcessError as e:
273             last_exc = e
274             if attempt_vf and vf_args:
275                 _err = (e.stderr.decode(errors="replace").strip()[-300:]
276                        if getattr(e, "stderr", None) else "")
277                 logger.warning(
278                     f"{segment_label}: watermark encode failed; retrying without it.
279 "
280                     f"ffmpeg: {_err or '(no stderr captured)'}"
281                 )
282                 continue
283             break
284         if os.path.exists(clip_path) and os.path.getsize(clip_path) > 0:
285             produced = True
286             if not attempt_vf and vf_args:
287                 logger.warning(f"{segment_label}: encoded WITHOUT the watermark (the
288 watermark filter failed).")
289             break
290         last_exc = None # ffmpeg succeeded but produced nothing
291         break
292
293     if produced:
294         return clip_path, None
295     elif last_exc is not None:
296         stderr_msg = last_exc.stderr.decode(errors='replace').strip()
297         logger.error(f"{segment_label}: FFmpeg trim failed.")
298         print(f" Command: {' '.join(cmd)}")
299         print(f" FFmpeg stderr: {stderr_msg[-500:]}") # Last 500 chars to avoid
300 flooding
301         return None, (idx, start, end, f"ffmpeg error: {stderr_msg[-200:]}")
302     else:
303         logger.error(f"{segment_label}: FFmpeg exited successfully but output clip
304 is missing or empty. Skipping.")
305         return None, (idx, start, end, "empty output clip")
306
307     def compile_highlights(self, raw_video_path: str, segments: List[Tuple[float,
308 float]], output_filename: str) -> str:
309         """
310         Extracts selected segments from a raw video and stitches them together
311         into a seamless highlights file, applying end_padding to prevent abrupt endings.
312         """
313         if not segments:
314             raise ValueError("No highlight segments provided to compile.")
315
316         # Never stitch the same footage twice: collapse overlapping/duplicate ranges
317         # (e.g. accumulated re-run segments or chunk-overlap dupes) before trimming.
318         original_count = len(segments)
319         segments = self._merge_overlapping(segments)

```

```

315         if len(segments) < original_count:
316             logger.info("Merged %d overlapping/duplicate segment(s) before stitching: %d
-> %d.",
317                         original_count - len(segments), original_count, len(segments))
318         if not segments:
319             raise ValueError("No valid (start < end) highlight segments after merge.")
320
321         if not os.path.exists(raw_video_path):
322             raise FileNotFoundError(f"[COMPILER ERROR] Source video file not found:
{raw_video_path}")
323
324         # Defense-in-depth: never let the output name escape output_dir (the API also
325         # validates this at the request boundary).
326         output_path = os.path.join(self.output_dir, os.path.basename(output_filename))
327
328         # Probe video duration so we can detect out-of-range segments
329         video_duration = self._get_video_duration(raw_video_path)
330         if video_duration > 0:
331             logger.info(f"Source video duration: {video_duration:.2f}s")
332         else:
333             logger.warning("Video duration unknown. Cannot validate segment boundaries
against video length.")
334
335         # We will create temporary clips and stitch them using FFmpeg concat demuxer
336         temp_clips = []
337         skipped_segments = []
338
339         # Create a temp directory within output_dir
340         with tempfile.TemporaryDirectory(dir=self.output_dir) as tmp_dir:
341             logger.info(f"Trimming {len(segments)} segments
(begin_padding={self.begin_padding}s, end_padding={self.end_padding}s)...")
342             # Build the (optional) watermark filtergraph once; it is applied during
every
343             # per-clip re-encode so the mark is baked into each clip identically.
344             watermark_filter = self._watermark_filter(tmp_dir)
345             if watermark_filter:
346                 logger.info("[watermark] applying branding overlay to each clip.")
347             for idx, (raw_start, raw_end) in enumerate(segments):
348                 clip_path, skip_record = self._trim_one_segment(
349                     idx, raw_start, raw_end, segments, video_duration,
350                     tmp_dir, raw_video_path, watermark_filter,
351                 )
352                 if clip_path is not None:
353                     temp_clips.append(clip_path)
354                 if skip_record is not None:
355                     skipped_segments.append(skip_record)
356
357             # Report summary of skipped segments
358             if skipped_segments:
359                 logger.info(f"\n[SUMMARY] {len(skipped_segments)} out of {len(segments)}
segments were SKIPPED:")
360                 for seg_idx, seg_start, seg_end, reason in skipped_segments:
361                     logger.info(f"    - Segment #{seg_idx+1} [{seg_start:.2f}s -
{seg_end:.2f}s]: {reason}")
362                 logger.info("")
363
364             if not temp_clips:
365                 raise RuntimeError(
366                     f"[COMPILER ERROR] Failed to generate ANY valid highlight clips. "
367                     f"All {len(segments)} segments were skipped. "
368                     f"Check the errors above for details. "
369                     f"Common causes: video file is corrupted, segments are out of range,
or ffmpeg is not installed."
370                 )
371

```

```

372         logger.info(f"Successfully trimmed {len(temp_clips)}/{len(segments)} clips.
Preparing to stitch...")
373
374         # Create a list file for FFmpeg concat demuxer
375         list_file_path = os.path.join(tmp_dir, "clips.txt")
376         # encoding pinned: a non-ASCII clip path would crash on Windows (cp1252
default);
377         # ffmpeg's concat demuxer reads the list as UTF-8 on every OS.
378         with open(list_file_path, "w", encoding="utf-8") as f:
379             for clip in temp_clips:
380                 # Escape single quotes in path
381                 safe_path = clip.replace("'", "'\\'")
382                 f.write(f"file '{safe_path}'\n")
383
384         logger.info("Stitching clips together...")
385         # Stitch them together without re-encoding (since they were encoded with
identical settings)
386         concat_cmd = [
387             "ffmpeg", "-y",
388             "-f", "concat",
389             "-safe", "0",
390             "-i", list_file_path,
391             "-c", "copy",
392             output_path
393         ]
394
395         try:
396             subprocess.run(concat_cmd, capture_output=True, check=True)
397
398             # Final verification
399             if not os.path.exists(output_path) or os.path.getsize(output_path) == 0:
400                 raise RuntimeError(
401                     f"[COMPILER ERROR] FFmpeg concatenation appeared to succeed, "
402                     f"but output file is missing or empty: {output_path}"
403                 )
404
405             output_size_mb = os.path.getsize(output_path) / (1024 * 1024)
406             logger.info(f"Dynamic highlight reel saved: {output_path}
({output_size_mb:.1f} MB)")
407             return output_path
408         except subprocess.CalledProcessError as e:
409             stderr_msg = e.stderr.decode(errors='replace').strip()
410             raise RuntimeError(
411                 f"[COMPILER ERROR] FFmpeg concatenation failed.\n"
412                 f"Command: {' '.join(concat_cmd)}\n"
413                 f"FFmpeg stderr: {stderr_msg[-500:]}"
414             ) from e
415

```

7. user (2026-06-21T14:02:02.106Z)

```

1     """Google Cloud Storage backend ? first-class object buckets.
2
3     Uses ``google-cloud-storage``, imported lazily so ``import backend.storage`` never pulls
it;
4     install with the ``storage-gcs`` extra. Credentials come from Application Default
Credentials
5     (``GOOGLE_APPLICATION_CREDENTIALS`` pointing at a service-account JSON, or workload
identity) ?
6     never from config. A ``client`` may be injected for tests.
7     """
8     from __future__ import annotations
9
10    from datetime import timedelta
11    from typing import Any, Iterator, Optional

```

```

12
13 from backend.storage.base import StorageBackend
14 from backend.storage.ref import StorageRef, StorageStat
15
16
17 class GCSStorage(StorageBackend):
18     def __init__(self, *, client: Any = None, project: Optional[str] = None,
19                 signed_url_ttl: int = 3600, **_ignored) -> None:
20         self._ttl = signed_url_ttl
21         if client is not None:
22             self._client = client
23         return
24         from google.cloud import storage # type: ignore # lazy: only when a gcs:// URI
is used
25         self._client = storage.Client(project=project)
26
27     def _blob(self, ref: StorageRef):
28         return self._client.bucket(ref.bucket).blob(ref.key)
29
30     def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
31         if dst is None:
32             raise ValueError("GCSStorage.fetch_to_local requires a dst path")
33         import os
34         dst = str(dst)
35         if os.path.exists(dst) and not overwrite:
36             return dst
37         os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
38         self._blob(ref).download_to_filename(dst)
39         return dst
40
41     def put(self, local_path, ref, *, content_type=None) -> StorageRef:
42         self._blob(ref).upload_from_filename(str(local_path), content_type=content_type)
43         return ref
44
45     def exists(self, ref) -> bool:
46         return bool(self._blob(ref).exists())
47
48     def stat(self, ref) -> StorageStat:
49         b = self._blob(ref)
50         b.reload()
51         return StorageStat(size=int(b.size or 0), modified=b.updated, etag=b.etag,
52                             content_type=b.content_type)
53
54     def list(self, prefix) -> Iterator[StorageRef]:
55         for blob in self._client.list_blobs(prefix.bucket, prefix=prefix.key):
56             yield StorageRef("gcs", prefix.bucket, blob.name,
57 f"gcs://{prefix.bucket}/{blob.name}")
58
59     def delete(self, ref) -> None:
60         self._blob(ref).delete()
61
62     def signed_url(self, ref, *, ttl=3600, method="GET") -> str:
63         return self._blob(ref).generate_signed_url(
64             version="v4", expiration=timedelta(seconds=ttl or self._ttl), method=method)

```

8. user (2026-06-21T14:02:02.526Z)

```

1     """Linux-native WASB shuttle-trajectory runner (no WSL).
2
3     The de-risk goal the ``DetectorRunner`` ABC was built for (see ``base.py`` ?
4     "Optional Linux-native path"; ``docs/DECOUPLED_COMPUTE/04-BUILD-AND-TEST-PLAN.md``
M3.5).
5     It is the sibling of :class:`~backend.pipeline.detectors.wasb_runner.WasbRunner` for a
6     Linux GPU box (GCP / a SaaS worker): it runs the same verified ``wasb_infer.py``

```

```

core
7         that the WSL adapter runs, so the trajectory CSV is identical (to a sub-pixel float
tolerance;
8         see the acceptance gate below) ? but WITHOUT the WSL transport:
9
10        * NO ``wslCommandMixin`` (no ``wsl -d <distro> bash -c``), NO ``conda activate`` (the
11        ``wasb`` env's python is invoked **by path**), NO ``/mnt/c`` path translation
12        (``to_wsl_mnt_path``). Paths are native; the video is already local on the box.
13        * ``device`` is ``cuda`` (default ? the WSL parity path), ``mps``, ``cpu``, or
``auto``
14        (M4: cuda-if-available-else-cpu ? the CPU fallback for a no-GPU box; explicit
``cuda`` stays
15        strict); threaded into ``wasb_infer.py --device`` (and so the Hydra
``runner.device`` override).
16        * weights / repo / python / scratch come from the **environment** first
17        (``WASB_WEIGHTS`` / ``WASB_REPO_DIR`` / ``WASB_PYTHON`` / ``WASB_SCRATCH``), then
18        ``indexing.wasb.*`` config, then a default ? so a box needs no config edits.
19
20        It emits the identical ``Frame,Visibility,X,Y`` CSV (same ``{stem}__{vid12}_wasb.csv``
21        name as the WSL runner) and reuses the model-agnostic ``parse_trajectory_csv`` /
22        ``trajectory_to_action_windows`` from ``tracknet_runner``, so the trajectory hybrids and
23        the windowing brain are unchanged. Selected via the existing ``detector_impl`` seam
24        (``trajectory_hybrid._resolve_runner`` ? ``_build_native_runner``);
``detector_impl="native"``.
25
26        **Acceptance gate (parity, NOT strict byte-equality):** the SAME clip through the WSL
runner
27        (Windows) and this runner (Linux, ``device=cuda``) yields the same trajectory CSV + the
same
28        candidate windows **within a small float tolerance (~1e-3 px)**. cuDNN is non-
deterministic
29        ACROSS GPUs, so a sub-pixel diff is expected on different hardware ? but it is byte-
exact
30        run-to-run on a FIXED GPU. ? Validated 2026-06-20 on an RTX 2070 (via WSL): native
streaming
31        vs the cached WSL-disk CSV matched **1194/1195** frames (the lone diff a ~5e-5 px edge
32        artifact), and two native runs were byte-identical (deterministic). This is a hardware
check
33        (a real GPU + the ``wasb`` env), so the offline, subprocess-mocked suite asserts the
contract,
34        not the numbers.
35        """
36        import os
37        import shutil
38        import logging
39        import subprocess
40        from dataclasses import dataclass
41        from typing import Any, Dict, List, Optional, Tuple
42
43        from backend.pipeline.detectors.base import DetectorRunner
44        from backend.pipeline.detectors.device import AUTO, VALID_DEVICES, DeviceContext
45        # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical to
WSL/TrackNet.
46        from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
47
48        logger = logging.getLogger(__name__)
49
50        # Path to the inference core we copy into the WASB repo's src/ (so it can import WASB's
51        # detectors/trackers/dataloaders + find configs/), exactly as the WSL adapter does.
52        _INFER_PY = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
53
54        # Single torch/cuda probe code, used by BOTH healthcheck and the lazy device-resolution
probe
55        # so the "cuda True/False" string they parse can never drift apart.
56        _CUDA_PROBE_CODE = "import torch; print('torch', torch.__version__, 'cuda',

```

```

torch.cuda.is_available())"
57
58
59     @dataclass
60     class NativeWasbConfig:
61         """Resolved settings for a Linux-native WASB run.
62
63         Built via :meth:`from_indexing_cfg` with **environment overrides** taking precedence
64         over ``indexing.wasb.*`` config (the box sets env; no config edits needed).
65         ``device`` defaults to ``cuda`` (the WSL parity path); ``stream_video`` defaults to True (the
66         perf win).
67         """
68         repo_dir: str = "~/models/WASB-SBDT" # native clone of nttcom/WASB-SBDT
69         python_bin: str = "python" # the `wasb` conda env python (invoked by
70         path, no activate)
71         weights_path: str = "" # REQUIRED (env WASB_WEIGHTS or
72         indexing.wasb.weights_path)
73         sport: str = "badminton"
74         device: str = "cuda" # auto | cuda | mps | cpu (auto = cuda-if-
75         available-else-cpu)
76         scratch_dir: str = "" # frame-extraction scratch (env); "" = next
77         to the output dir
78         timeout_sec: int = 1800 # 30 min; a hung GPU call is killed (0 = no
79         timeout)
80         keep_frames: bool = False # retain the decoded frame cache after
81         success (debug only)
82         # NATIVE DEFAULT = STREAMING: skip the cv2 PNG extraction that dominates a long
83         clip's
84         # wall-clock (measured ~50s on a 20s clip) ? bit-identical to disk (verified on a
85         real GPU
86         # 2026-06-20). The GPU box reads from a local NVMe so streaming is the right default
87         here.
88         stream_video: bool = True
89
90     @classmethod
91     def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "NativeWasbConfig":
92         w = (idx_cfg or {}).get("wasb", {}) or {}
93         env = os.environ.get
94         # stream_video is tri-state in config: None/absent => native default (stream);
95         an
96         # explicit True/False still wins (e.g. False for a container cv2 can't seek).
97         sv = w.get("stream_video")
98         return cls(
99             repo_dir=env("WASB_REPO_DIR") or w.get("repo_dir", "~/models/WASB-SBDT"),
100             python_bin=env("WASB_PYTHON") or w.get("python_bin", "python"),
101             weights_path=env("WASB_WEIGHTS") or w.get("weights_path", "") or "",
102             sport=w.get("sport", "badminton"),
103             device=env("WASB_DEVICE") or w.get("device", "cuda"),
104             scratch_dir=env("WASB_SCRATCH") or env("RALLY_SCRATCH") or env("TMPDIR") or
105             "",
106             timeout_sec=int(w.get("timeout_sec", 1800)),
107             keep_frames=bool(w.get("keep_frames", False)),
108             stream_video=(True if sv is None else bool(sv)),
109         )
110
111     class NativeWasbRunner(DetectorRunner):
112         """WASB runner that executes natively on a Linux GPU box ? no WSL. See module
113         docstring."""
114
115         def __init__(self, cfg: NativeWasbConfig):
116             self.cfg = cfg
117             # Cached CUDA-availability probe (used to resolve device="auto"). Set by
118             healthcheck;

```



```

106         # lazily probed by run_predict if healthcheck was skipped. None = not yet
probed.
107         self._cuda_available: Optional[bool] = None
108
109         # --- low-level native exec (the single place tests patch)
-----
110         def _run(self, argv: List[str], cwd: Optional[str] = None) ->
subprocess.CompletedProcess:
111             logger.debug("native exec: %s (cwd=%s)", " ".join(argv), cwd)
112             return subprocess.run(argv, cwd=cwd, capture_output=True, text=True,
113                                   timeout=(self.cfg.timeout_sec or None))
114
115         # --- device resolution (M4 DeviceContext: device="auto" ? cuda-if-available-else-
cpu) -----
116         def _probe_cuda(self) -> bool:
117             """Probe ``torch.cuda.is_available()`` in the wasb env's python (a subprocess).
Any failure
118             (missing python / timeout / import error) is treated as 'no CUDA'."""
119             try:
120                 res = self._run([self.cfg.python_bin, "-c", _CUDA_PROBE_CODE])
121             except (FileNotFoundError, subprocess.TimeoutExpired):
122                 return False
123             return res.returncode == 0 and "cuda True" in (res.stdout or "")
124
125         def _cuda_is_available(self) -> bool:
126             """CUDA availability, cached across healthcheck + run_predict so we probe at
most once."""
127             if self._cuda_available is None:
128                 self._cuda_available = self._probe_cuda()
129             return self._cuda_available
130
131         def _effective_device(self) -> str:
132             """The torch device to actually pass to ``wasb_infer.py``. Only ``auto`` needs
the CUDA
133             probe; an explicit cuda/mps/cpu resolves to itself (so ``device='cuda'`` is
unchanged)."""
134             if self.cfg.device == AUTO:
135                 return DeviceContext(AUTO, self._cuda_is_available()).effective
136             return self.cfg.device
137
138         def _repo_src(self) -> str:
139             return os.path.join(os.path.expanduser(self.cfg.repo_dir), "src")
140
141         def _copy_infer_script(self) -> bool:
142             """Copy wasb_infer.py into the WASB repo src/ so it imports WASB modules + finds
configs/."""
143             repo_src = self._repo_src()
144             try:
145                 os.makedirs(repo_src, exist_ok=True)
146                 shutil.copy(_INFER_PY, os.path.join(repo_src, "wasb_infer.py"))
147                 return True
148             except OSError as e:
149                 logger.error("Failed to copy wasb_infer.py into %s: %s", repo_src, e)
150             return False
151
152         def _rm_rf(self, path: Optional[str]) -> None:
153             """Best-effort recursive delete of a native path (post-success cleanup; never
raises)."""
154             if path:
155                 shutil.rmtree(path, ignore_errors=True)
156
157         def healthcheck(self) -> Tuple[bool, str]:
158             """Verify weights + repo present and the `wasb` python imports torch (and sees a
GPU on cuda)."""
159             if not self.cfg.weights_path:

```

```

160         return False, ("WASB weights path is empty ? set WASB_WEIGHTS (or
indexing.wasb.weights_path).")
161     weights = os.path.expanduser(self.cfg.weights_path)
162     if not os.path.exists(weights):
163         return False, f"WASB weights not found at {weights}"
164     repo_src = self._repo_src()
165     if not os.path.isdir(repo_src):
166         return False, (f"WASB-SBDT repo src/ not found at {repo_src} ? clone
nttcom/WASB-SBDT "
167                        f"(set WASB_REPO_DIR / indexing.wasb.repo_dir).")
168     # Fail fast on a typo'd device (the device-policy seam) ? a clear error here
beats a cryptic
169     # argparse rc=2 from wasb_infer.py at run time.
170     if self.cfg.device not in VALID_DEVICES:
171         return False, (f"unknown device {self.cfg.device!r} ? valid: {'',
'.join(VALID_DEVICES)}").")
172     try:
173         res = self._run([self.cfg.python_bin, "-c", _CUDA_PROBE_CODE])
174     except FileNotFoundError:
175         return False, f"python binary not found: {self.cfg.python_bin!r} (set
WASB_PYTHON).")
176     except subprocess.TimeoutExpired:
177         return False, "native torch healthcheck timed out."
178     out = (res.stdout or "").strip()
179     if res.returncode != 0:
180         return False, f"`${self.cfg.python_bin}` torch import failed: {(res.stderr or
out).strip()}"
181     # M4: resolve the device. Cache the probe so run_predict reuses it (no second
subprocess).
182     self._cuda_available = "cuda True" in out
183     ctx = DeviceContext(self.cfg.device, self._cuda_available)
184     if ctx.cuda_required_but_missing:
185         # Unchanged pre-M4 behaviour for an EXPLICIT cuda request: hard-fail, never
a silent
186         # slow-CPU downgrade. Set indexing.wasb.device=auto for a CPU fallback.
187         return False, f"device=cuda but torch.cuda.is_available() is False ({out})"
188     if ctx.fell_back:
189         logger.warning("device=auto: no CUDA GPU detected (%s) ? FALLING BACK TO
CPU; WASB "
190                        "inference will be slow. Set indexing.wasb.device=cuda to
require a GPU.", out)
191     return True, out
192
193     def run_predict(self, video_win_path: str, output_win_dir: str,
194                    video_id: Optional[str] = None) -> Optional[str]:
195         """Run WASB natively on a video. Returns the path to the trajectory CSV, or
None.
196
197         Output artifacts are namespaced by ``video_id`` (the file MD5) exactly as the
WSL
198         runner does, and the CSV is named ``{stem}__{vid12}_wasb.csv`` ? byte-for-byte
the
199         same downstream contract, so the parity comparison is apples-to-apples.
200         """
201         output_dir = os.path.abspath(output_win_dir)
202         os.makedirs(output_dir, exist_ok=True)
203         # Normalize for a cross-platform basename (tests pass Windows paths on Linux).
204         norm = str(video_win_path).replace("\\", "/")
205         stem, _ext = os.path.splitext(os.path.basename(norm))
206         # Content-derived key: same filename + different bytes -> different key (no
cache reuse).
207         key = f"{stem}__{video_id[:12]}" if video_id else (stem or "wasb")
208         out_csv = os.path.join(output_dir, f"{key}_wasb.csv")
209
210         if not self._copy_infer_script():

```

```

211         logger.error("Could not stage wasb_infer.py into the WASB repo src/.")
212         return None
213
214     weights = os.path.expanduser(self.cfg.weights_path)
215     argv = [self.cfg.python_bin, "wasb_infer.py",
216            "--video", os.path.abspath(str(video_win_path))]
217     frames_dir: Optional[str] = None
218     if self.cfg.stream_video:
219         # Fast path: decode in-process, no PNG extraction. Output is bit-identical
to disk.
220         argv.append("--stream-video")
221     else:
222         scratch = self.cfg.scratch_dir or output_dir
223         frames_dir = os.path.join(scratch, f"{key}_frames")
224         argv += ["--frames_out_dir", frames_dir]
225         # M4: resolve device="auto" to the effective device (cuda-if-available-else-
cpu); an
226         # explicit cuda/mps/cpu passes through unchanged. Never sends "auto" to
wasb_infer.py.
227         device = self._effective_device()
228         argv += ["--weights", weights,
229                "--sport", self.cfg.sport,
230                "--device", device,
231                "--out", out_csv]
232
233         logger.info("Running native WASB inference (device=%s%s) on %s ...",
234                    device, " [auto?fallback]" if self.cfg.device == AUTO and device !=
"cuda" else "",
235                    os.path.basename(norm))
236         try:
237             res = self._run(argv, cwd=self._repo_src())
238         except FileNotFoundError:
239             logger.error("python binary not found: %r (set WASB_PYTHON).",
self.cfg.python_bin)
240             return None
241         except subprocess.TimeoutExpired:
242             logger.error("WASB native inference timed out after %ss.",
self.cfg.timeout_sec)
243             return None
244         if res.returncode != 0:
245             logger.error("WASB native inference failed:\n%s", (res.stderr or res.stdout
or "")[-2000:])
246             return None
247         if not os.path.exists(out_csv):
248             logger.error("WASB native finished but expected CSV not found at %s",
out_csv)
249             return None
250         logger.info("WASB trajectory CSV: %s", out_csv)
251
252         # Success ? the CSV is the durable output; the decoded frame cache is a pure
intermediate.
253         # On any earlier failure we return above WITHOUT cleaning, so the cache survives
a resume.
254         # (Streaming never materializes frames, so there is nothing to drop on that
path.)
255         if not self.cfg.keep_frames and frames_dir:
256             logger.info("Cache hygiene: removing frame cache for %s "
257                        "(set indexing.wasb.keep_frames=true to retain).", key)
258             self._rm_rf(frames_dir)
259         return out_csv
260
261     # Trajectory parsing + windowing are inherited from tracknet_runner (model-
agnostic).
262     parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
263     trajectory_to_action_windows =

```

```
staticmethod(TrackNetRunner.trajectory_to_action_windows)
264
```

9. assistant (2026-06-21T14:02:08.273Z)

The codebase claims are verifiable. Let me note discrepancies in line numbers and check exact references. The claim says `backend/utils/video.py` line 11 for `ffprobe` ? it's line 12 (line 11 is the `subprocess.check_output([` opener, line 12 is the actual "ffprobe" string). And compiler.py line 113 for ffprobe ? line 113 is "ffprobe", "-v", "error",` . Let me verify the line references more carefully, plus check the GCS storage claim about whether fetch_to_local requires google-cloud-storage (relevant to the recommendation's "no google-cloud-storage dependency" claim).`

Now let me do the web research on the external facts. Let me load the `WebSearch` and `WebFetch` tools.

10. assistant (2026-06-21T14:02:11.915Z)

Now let me research the external facts. I'll batch several independent searches.

11. user (2026-06-21T14:02:20.704Z)

Web search results for query: "imageio-ffmpeg bundled ffmpeg binary license GPL libx264 ffprobe not included"

Links: [{"title": "GitHub - imageio/imageio-ffmpeg: FFMPEG wrapper for Python · GitHub", "url": "https://github.com/imageio/imageio-ffmpeg"}, {"title": "FFmpeg License and Legal Considerations", "url": "https://www.ffmpeg.org/legal.html"}, {"title": "FFmpeg: License", "url": "https://ffmpeg.org/doxygen/3.4/md_LICENSE.html"}, {"title": "imageio.plugins.ffmpeg ? imageio 2.11.0 documentation", "url": "https://imageio.readthedocs.io/en/v2.11.0/reference/_backends/imageio.plugins.ffmpeg.html"}, {"title": "imageio.plugins.ffmpeg ? imageio 2.37.3 documentation", "url": "https://imageio.readthedocs.io/en/stable/_autosummary/imageio.plugins.ffmpeg.html"}, {"title": "imageio-ffmpeg · PyPI", "url": "https://pypi.org/project/imageio-ffmpeg/"}]

Based on the search results, here's what I found about `imageio-ffmpeg` and the related licensing considerations:

imageio-ffmpeg and FFmpeg Binary Distribution

`imageio-ffmpeg` is a simple and reliable `ffmpeg` wrapper for working with video files that publishes platform-specific wheels that include the binary `ffmpeg` executables. The platform-specific wheels contain the binary executable of `ffmpeg`, which makes this package around 60 MiB in size.

FFmpeg Licensing and GPL

The `FFmpeg` licensing situation is complex regarding `GPL` and `libx264`:

`FFmpeg` incorporates several optional parts and optimizations that are covered by the GNU General Public License (`GPL`) version 2 or later, and if those parts get used the `GPL` applies to all of `FFmpeg`. One should make sure their program is not using any `GPL` libraries (notably `libx264`).

When `FFmpeg` is compiled with certain `GPL` components like `libx264` enabled, the resulting binary falls under `GPL` licensing constraints, which can have implications for redistribution and usage.

About ffprobe

The search results did not contain specific information about `ffprobe` being included or not included with `imageio-ffmpeg`. You may want to consult the `imageio-ffmpeg` GitHub repository directly or the project's documentation for details about what tools are bundled with the package.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

12. user (2026-06-21T14:02:23.410Z)

PyPI Storage Limits

****Default Limits:****

The documentation states: "The current default limits are ****100.0 MB**** for individual files and ****10.0 GB**** for the entire project." So yes, the per-file limit is 100 MB and the project total is 10 GB (not MiB/GiB, but MB/GB as specified).

****Process for Requesting a File Size Increase:****

Yes, an existing release is required first. The guidelines specify: "Make sure you've uploaded at least one release for the project that's under the limit (a developmental release version number is fine)." Then you file an issue with:

- A link to your project
- The desired file size in megabytes
- Which index (PyPI, TestPyPI, or both)
- A brief project description explaining the need

****Process for Requesting a Project Size Increase:****

The documentation recommends first removing unnecessary releases or files. If that's insufficient, you file an issue providing:

- A link to your project
- Total project size in gigabytes
- A description of your project and reason for additional storage

No explicit requirement for an existing release under the limit is mentioned for project-level increases, though the implication is that you should exhaust alternatives before requesting an increase.

13. assistant (2026-06-21T14:02:28.053Z)

Important nuance found: PyPI doc says ****100.0 MB**** and ****10.0 GB**** (decimal MB/GB) not MiB/GiB. The claim says "100 MiB" and "10 GiB". Let me verify the imageio-ffmpeg specifics (license, ffprobe absence, binary resolution order, default codec) directly from the repo, and check the WASB-SBDT license and other ffmpeg build facts.

14. user (2026-06-21T14:02:33.458Z)

imageio-ffmpeg Analysis

****License:**** BSD-2-Clause

****ffprobe bundling:**** The documentation does not specify whether ffprobe is included?only ffmpeg is mentioned.

****Default MP4 codec:**** The library states the default is "libx264 for .mp4 (if available from the ffmpeg executable)."

****Binary resolution order:**** According to the documentation, the priority is: "the binary defined by the IMAGEIO_FFMPEG_EXE environment variable, the binary distributed with imageio-ffmpeg, an ffmpeg binary installed with conda, or the system ffmpeg (in that order)."

****GPL status:**** The page does not disclose whether the bundled ffmpeg is a GPL build or provide licensing details about the included binary.

15. user (2026-06-21T14:02:35.122Z)

WASB-SBDT Repository License and Model Weights

****License:**** Yes, the repository is MIT licensed, as indicated by the badge stating "License: MIT" and the LICENSE.md file in the repository.

****Pretrained Weights:**** The repository does provide pretrained weights. According to the news section, "evaluation codes of pretrained WASB models" are available, along with evaluation codes for other models like DeepBall, TrackNetV2, and ResTrackNetV2.

****MODEL_Z00.md:**** Yes, a MODEL_Z00.md file exists in the repository, referenced in the "See MODEL_Z00.md for available model weights" instruction.

****Hosting Location:**** The provided content does not specify where the weights are hosted or mention Google Drive. You would need to consult the MODEL_Z00.md file directly to determine the hosting location and download instructions for the pretrained models.

16. assistant (2026-06-21T14:02:40.037Z)

Good. The imageio-ffmpeg wrapper facts are confirmed: BSD-2-Clause, libx264 default for mp4, resolution order matches. Now let me check the WASB MODEL_Z00.md hosting (Google Drive?), the imageio-ffmpeg ffprobe issue #23, the LGPL/GPL ffmpeg build facts (BtbN, gyan.dev, libopenh264 BSD), and the PyInstaller CUDA torch bundle size claim.

17. user (2026-06-21T14:02:44.693Z)

Pretrained Model Weights Hosting

The pretrained model weights are hosted on ****Google Drive****. According to the MODEL_Z00.md file, all models across different sports?including badminton?are made available through individual Google Drive links rather than being stored directly in the repository.

For badminton specifically, the document provides Google Drive links for seven different methods:

- DeepBall
- DeepBall-Large
- BallSeg
- TrackNetV2
- ResTrackNetV2
- MonoTrack
- WASB (the authors' method)

The same hosting approach applies to all other sports covered (soccer, tennis, volleyball, and basketball), with each method-sport combination having its own dedicated Google Drive link.

18. user (2026-06-21T14:02:45.004Z)

Summary of Issue #23: ffprobe Availability

The issue does ****not**** confirm that imageio-ffmpeg bundles only ffmpeg and not ffprobe. Rather, it requests adding ffprobe functionality.

User thenewguy states: "It would be handy to use FFPROBE for metadata," indicating that ffprobe support was either missing or had been lost at that time. The issue references a related discussion from the moviepy project, suggesting this was a broader feature request across dependent libraries.

The issue itself is a feature request, not a confirmation of what is or isn't bundled with the library.

19. user (2026-06-21T14:02:47.467Z)

Web search results for query: "PyInstaller pytorch CUDA cudnn bundle size GB cannot split CPU-only smaller"

Links: [{"title": "Reducing the Size of PyTorch-Based Executable · pyinstaller · Discussion"}]

```
#8552","url":"https://github.com/orgs/pyinstaller/discussions/8552"},"title":"PyInstaller,
issue to build a package with CUDA, CuDNN and Tensorflow on Windows 10 · Issue #7175 · pyinstall
er/pyinstaller","url":"https://github.com/pyinstaller/pyinstaller/issues/7175"},"title":"Reduci
ng the Size of PyTorch-Based Executable · Issue #8551 · pyinstaller/pyinstaller","url":"https://
github.com/pyinstaller/pyinstaller/issues/8551"},"title":"Error for binaries larger than 2Gb ·
Issue #3939 ·
pyinstaller/pyinstaller","url":"https://github.com/pyinstaller/pyinstaller/issues/3939"]}]
```

Based on the search results, I found relevant information about your query regarding PyInstaller, PyTorch, CUDA, and cuDNN bundle sizes:

The original torch2.3+cu118 in conda is around 5GB, and the torch library in the app is about 4GB. The key issue is that most of that size are binary extensions and CUDA/cuDNN shared libs, which cannot be split.

This is a known limitation when bundling PyTorch with GPU support (CUDA and cuDNN) using PyInstaller. The binary extensions and GPU libraries are large and cannot be easily separated or reduced in size during the bundling process. If you're looking to create smaller executables, one approach would be to use a CPU-only version of PyTorch, which would be significantly smaller than the GPU-enabled version.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

20. assistant (2026-06-21T14:02:51.917Z)

Key finding on the imageio-ffmpeg ffprobe issue #23: the cited source does NOT confirm "bundles only ffmpeg, not ffprobe" ? it's a feature request asking for ffprobe support. The underlying claim (no ffprobe bundled) is likely true, but the cited source doesn't directly state it. Let me verify the ffprobe-absence claim from a more authoritative angle and confirm the PyInstaller size numbers, plus the Btbn/gyan.dev GPL and libopenh264 BSD facts.

21. user (2026-06-21T14:02:56.947Z)

PyTorch PyInstaller Bundle Size Questions

Based on the discussion, here are answers to your questions:

****Reported bundle size with CUDA:****

Yes, the maintainer reports "Even onefile build with pip-installed torch2.3+cu118 seems to come out at 2.6GB." The original user noted their torch library alone was approximately 4GB in the packaged app.

****Why CUDA/cuDNN libs can't be split:****

The maintainer states these are "binary extensions and CUDA/cuDNN shared libs, which cannot be split." This means the compiled libraries and dependencies are monolithic and cannot be meaningfully reduced through standard packaging techniques.

****CPU-only torch size:****

The maintainer suggests this as a potential solution, implying CPU-only builds would be significantly smaller than CUDA-enabled versions. However, the exact size isn't specified in this discussion. The maintainer notes that smaller executables they've seen either use "CPU-only `torch` build (no CUDA), or some older build with older (and smaller) CUDA libs."

****Key takeaway:**** The maintainer emphasizes that "Those two aspects are mutually exclusive" ? using PyTorch while maintaining a small executable size are fundamentally incompatible goals, especially with CUDA support.

22. user (2026-06-21T14:02:58.394Z)

Web search results for query: "imageio-ffmpeg does not include ffprobe only ffmpeg binary"

Links: [{"title":"GitHub - imageio/imageio-ffmpeg: FFMPEG wrapper for Python ·

```
GitHub", "url": "https://github.com/imageio/imageio-ffmpeg"}, {"title": "ffprobe · Issue #23 · imageio/imageio-ffmpeg", "url": "https://github.com/imageio/imageio-ffmpeg/issues/23"}, {"title": "imageio.plugins.ffmpeg ? imageio 2.10.0 documentation", "url": "https://imageio.readthedocs.io/en/v2.10.0/reference/_backends/imageio.plugins.ffmpeg.html"}, {"title": "imageio-ffmpeg · PyPI", "url": "https://pypi.org/project/imageio-ffmpeg/"}, {"title": "ffmpeg - Alpine Linux packages", "url": "https://pkgs.alpinelinux.org/package/edge/community/x86/ffmpeg"}, {"title": "How to use the imageio.plugins.ffmpeg function in ImageIO | Snyc", "url": "https://snyc.io/advisor/python/imageio/functions/imageio.plugins.ffmpeg"}, {"title": "make ffmpeg's can_read more sophisticated · Issue #686 · imageio/imageio", "url": "https://github.com/imageio/imageio/issues/686"}, {"title": "(re)new plugin for video · Issue #406 · imageio/imageio", "url": "https://github.com/imageio/imageio/issues/406"}, {"title": "imageio.plugins.ffmpeg ? imageio 2.37.3 documentation", "url": "https://imageio.readthedocs.io/en/stable/_autosummary/imageio.plugins.ffmpeg.html"}]
```

Based on the search results, I found information about imageio-ffmpeg and its handling of ffmpeg and ffprobe binaries.

imageio-ffmpeg publishes platform-specific wheels that include the binary ffmpeg executables, and the platform-specific wheels contain the binary executable of ffmpeg.

However, the search results show that ffprobe is not included in imageio-ffmpeg. This is evident from GitHub issue #23 on the imageio-ffmpeg repository, where a user requested the addition of ffprobe functionality for metadata-related operations, suggesting that it was not then (and likely still is not) included by default.

If you need ffprobe functionality alongside imageio-ffmpeg, you would need to either:

1. Install ffmpeg from your system package manager (which includes both ffmpeg and ffprobe), or
2. Use the `IMAGEIO\_FFMPEG\_EXE` environment variable to point to a custom ffmpeg installation that includes ffprobe

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

23. user (2026-06-21T14:03:00.937Z)

Web search results for query: "libopenh264 license BSD Cisco OpenH264 FFmpeg LGPL build H.264 encoder"

```
Links: [{"title": "GitHub - cisco/openh264: Open Source H.264 Codec · GitHub", "url": "https://github.com/cisco/openh264"}, {"title": "GitHub - Crigges/Prebuilt-LGPL-2.1-FFmpeg-with-OpenH264: Prebuilt LGPL v2.1 MinGW cross compiled binaries for Windows 64bit with OpenH264 for H264 encoding · GitHub", "url": "https://github.com/Crigges/Prebuilt-LGPL-2.1-FFmpeg-with-OpenH264"}, {"title": "FFmpeg Commercial License Guide: LGPL, GPL & Patent Risks | 32blog", "url": "https://32blog.com/en/ffmpeg/ffmpeg-commercial-license-guide"}, {"title": "OpenH264 download | SourceForge.net", "url": "https://sourceforge.net/projects/openh264.mirror/"}, {"title": "OpenH264 - Wikipedia", "url": "https://en.wikipedia.org/wiki/OpenH264"}, {"title": "Openh264", "url": "https://www.openh264.org/BINARY_LICENSE.txt"}, {"title": "OpenH264 - Fedora Project Wiki", "url": "https://fedoraproject.org/wiki/OpenH264"}]
```

Based on the search results, here's what I found about libopenh264, licenses, and FFmpeg integration:

License Information

OpenH264 is licensed under BSD, and it is released under the terms of the Simplified BSD License.

Cisco's Patent Coverage

Cisco has taken their H.264 implementation and open-sourced it under BSD license terms. Notably, Cisco will not pass on its MPEG-LA licensing costs for this module, and based on the current

licensing environment, this will effectively make H.264 free for use on supported platforms.

LGPL Compatibility with FFmpeg

OpenH264 works within an LGPL build of FFmpeg. FFmpeg supports OpenH264 as both an encoder and decoder. To build FFmpeg with OpenH264 support, you can use the `--enable-libopenh264` configuration flag.

Important Note About Source vs. Binary

There's a distinction regarding patent coverage: projects that use Cisco's source code instead of its binaries would be legally responsible for paying all royalties to MPEG LA themselves, while those using Cisco's precompiled binaries receive patent coverage at no cost.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

24. assistant (2026-06-21T14:03:05.085Z)

Now let me verify the remaining ffmpeg-build facts: BtbN GPL builds being GPLv3 with libx264/libx265/ffprobe (issue #29), BtbN LGPL builds omitting libx264, gyan.dev GPLv3 builds, and the FFmpeg legal.html GPL-propagation/redistribution facts. Let me also confirm huggingface_hub Apache-2.0 + hf_hub_download features.

25. user (2026-06-21T14:03:11.980Z)

Response

Based on the webpage content provided, I cannot answer your questions. The page displays a GitHub issue titled "What's the difference between ffmpeg gpl version and lgpl version?" but **no actual answer or discussion content is shown**. The issue was opened on December 1, 2020, with "No description provided."

The page contains only GitHub's navigation elements and metadata?not the substantive technical information needed to address your questions about:

- GPLv3 specifications in BtbN FFmpeg builds
- Inclusion of libx264, libx265, or ffprobe
- Specific differences between GPL and LGPL build variants

To get accurate answers, you would need to access the actual comments and responses within that GitHub issue thread.

26. user (2026-06-21T14:03:12.234Z)

FFmpeg Licensing Analysis

****Does --enable-gpl make the whole build GPL?****

Yes. The page states: "FFmpeg incorporates several optional parts and optimizations that are covered by the GNU General Public License (GPL) version 2 or later. If those parts get used the GPL applies to all of FFmpeg."

****What about libx264 requiring GPL?****

The document explicitly warns: "Make sure your program is not using any GPL libraries (notably libx264)." This indicates libx264 is GPL-licensed and incompatible with LGPL-only compliance strategies.

****LGPL redistribution obligations:****

For LGPL-compliant builds, the checklist requires:

- Use dynamic linking to FFmpeg libraries
- Distribute FFmpeg source code matching your binaries
- Host sources on the same server as binaries
- Include prominent attribution on download pages and in your application

****Can GPL FFmpeg be embedded in proprietary products?****

The document states: "FFmpeg is not available under any other licensing terms, especially not proprietary/commercial ones, not even in exchange for payment." A GPL-built FFmpeg cannot be embedded in proprietary redistributed products without making those products GPL-licensed as well?the GPL's viral nature prevents this approach.

27. user (2026-06-21T14:03:14.604Z)

huggingface_hub Package Overview

****License:**** The package is licensed under the "Apache Software License (Apache-2.0)" as specified in the PyPI metadata.

****hf_hub_download Function:**** Yes, the documentation explicitly mentions this capability. The quick start guide shows: `hf_hub_download(repo_id="tiiuae/falcon-7b-instruct", filename="config.json")` for downloading individual files.

****Cache and Download Features:**** The package supports content-addressed local caching. The documentation states: "Files will be downloaded in a local cache folder. More details in [this guide]" regarding cache management. Additionally, the library provides `snapshot_download()` for downloading entire repositories, which works with the local caching system.

****Offline Mode:**** While the PyPI page doesn't explicitly detail the `HF_HUB_OFFLINE` environment variable or `local_files_only` parameter in the visible content, the key features section emphasizes download and management capabilities consistent with offline functionality through cached files.

28. assistant (2026-06-21T14:03:18.050Z)

Most facts confirmed. The BtbN issue #29 fetch didn't return content. Let me verify the BtbN GPL/LGPL build distinction and gyan.dev builds directly, and confirm huggingface_hub offline switch + resumable downloads via the official docs.

29. user (2026-06-21T14:03:21.073Z)

Downloading files

Download a single file

hf_hub_download[[huggingface_hub.hf_hub_download]]

```
#### huggingface_hub.hf_hub_download[[huggingface_hub.hf_hub_download]]
```

[Source](https://github.com/huggingface/huggingface_hub/blob/v1.20.1/src/huggingface_hub/file_download.py#L829)

Download a given file if it's not already present in the local cache.

The new cache file layout looks like this:

- The cache directory contains one subfolder per repo_id (namespaced by repo type)
- inside each repo folder:
 - refs is a list of the latest known revision => commit_hash pairs
 - blobs contains the actual file blobs (identified by their git-sha or sha256, depending on whether they're LFS files or not)
 - snapshots contains one subfolder per commit, each "commit" contains the subset of the files that have been resolved at that particular commit. Each filename is a symlink to the blob

at that particular commit.

...

```
[ 96] .
??? [ 160] models--julien-c--EsperBERTo-small
      ??? [ 160] blobs
        ?   ??? [321M] 403450e234d65943a7dcf7e05a771ce3c92faa84dd07db4ac20f592037a1e4bd
        ?   ??? [ 398] 7cb18dc9bafbfcf74629a4b760af1b160957a83e
        ?   ??? [1.4K] d7edf6bd2a681fb0175f7735299831ee1b22b812
      ??? [ 96] refs
        ?   ??? [ 40] main
      ??? [ 128] snapshots
        ??? [ 128] 2439f60ef33a0d46d85da5001d52aeda5b00ce9f
        ?   ??? [ 52] README.md -> ../../blobs/d7edf6bd2a681fb0175f7735299831ee1b22b812
        ?   ??? [ 76] pytorch_model.bin ->
../../blobs/403450e234d65943a7dcf7e05a771ce3c92faa84dd07db4ac20f592037a1e4bd
      ??? [ 128] bbc77c8132af1cc5cf678da3f1ddf2de43606d48
        ??? [ 52] README.md -> ../../blobs/7cb18dc9bafbfcf74629a4b760af1b160957a83e
        ??? [ 76] pytorch_model.bin ->
../../blobs/403450e234d65943a7dcf7e05a771ce3c92faa84dd07db4ac20f592037a1e4bd
...
```

If ``local_dir`` is provided, the file structure from the repo will be replicated in this location. When using this option, the ``cache_dir`` will not be used and a ``.cache/huggingface/`` folder will be created at the root of ``local_dir`` to store some metadata related to the downloaded files. While this mechanism is not as robust as the main cache-system, it's optimized for regularly pulling the latest version of a repository.

****Parameters:****

`repo_id (`str`)` : A user or an organization name and a repo name separated by a ``/``.

`filename (`str`)` : The name of the file in the repo.

`subfolder (`str`, *optional*)` : An optional value corresponding to a folder inside the model repo.

`repo_type (`str`, *optional*)` : Set to ``"dataset"``, ``"space"`` or ``"kernel"`` if downloading from a dataset, space or kernel repo, ``None`` or ``"model"`` if downloading from a model. Default is ``None``.

`revision (`str`, *optional*)` : An optional Git revision id which can be a branch name, a tag, or a commit hash.

`library_name (`str`, *optional*)` : The name of the library to which the object corresponds.

`library_version (`str`, *optional*)` : The version of the library.

`cache_dir (`str`, `Path`, *optional*)` : Path to the folder where cached files are stored.

`local_dir (`str` or `Path`, *optional*)` : If provided, the downloaded file will be placed under this directory.

`user_agent (`dict`, `str`, *optional*)` : The user-agent info in the form of a dictionary or a string.

`force_download (`bool`, *optional*, defaults to `False`)` : Whether the file should be downloaded even if it already exists in the local cache.

`etag_timeout (`float`, *optional*, defaults to `10`)` : When fetching ETag, how many seconds to wait for the server to send data before giving up which is passed to ``requests.request``.

`token (`str`, `bool`, *optional*)` : A token to be used for the download. - If ``True``, the token

is read from the HuggingFace config folder. - If a string, it's used as the authentication token.

`local_files_only` (`bool`, *optional*, defaults to `False`) : If `True`, avoid downloading the file and return the path to the local cached file if it exists.

`headers` (`dict`, *optional*) : Additional headers to be sent with the request.

`endpoint` (`str`, *optional*) : The Hub endpoint to send the request to. Defaults to the value of `HF\_ENDPOINT`.

`tqdm_class` (`tqdm`, *optional*) : If provided, overwrites the default behavior for the progress bar. Passed argument must inherit from `tqdm.auto.tqdm` or at least mimic its behavior. Defaults to the custom HF progress bar that can be disabled by setting `HF\_HUB\_DISABLE\_PROGRESS\_BARS` environment variable.

`dry_run` (`bool`, *optional*, defaults to `False`) : If `True`, perform a dry run without actually downloading the file. Returns a `[DryRunFileInfo](/docs/huggingface_hub/v1.20.1/en/package_reference/hf_api#huggingface_hub.DryRunFileInfo)` object containing information about what would be downloaded.

****Returns:****

`str` or `[DryRunFileInfo](/docs/huggingface_hub/v1.20.1/en/package_reference/hf_api#huggingface_hub.DryRunFileInfo)`

- If `dry_run=False`: Local path of file or if networking is off, last version of file cached on disk.

- If `dry_run=True`: A `[DryRunFileInfo](/docs/huggingface_hub/v1.20.1/en/package_reference/hf_api#huggingface_hub.DryRunFileInfo)` object containing download information.

hf_hub_url[[huggingface_hub.hf_hub_url]]

`huggingface_hub.hf_hub_url[[huggingface_hub.hf_hub_url]]`

[Source](https://github.com/huggingface/huggingface_hub/blob/v1.20.1/src/huggingface_hub/file_download.py#L197)

Construct the URL of a file from the given information.

The resolved address can either be a huggingface.co-hosted url, or a link to Cloudfront (a Content Delivery Network, or CDN) for large files which are more than a few MBs.

Example:

```
```python
>>> from huggingface_hub import hf_hub_url

>>> hf_hub_url(
... repo_id="julien-c/EsperBERTo-small", filename="pytorch_model.bin"
...)
'https://huggingface.co/julien-c/EsperBERTo-small/resolve/main/pytorch_model.bin'
```
```

> [!TIP]

> Notes:

>

> Cloudfront is replicated over the globe so downloads are way faster for the end user (and it also lowers our bandwidth costs).

>

> Cloudfront aggressively caches files by default (default TTL is 24 hours), however this is not an issue here because we implement a git-based versioning system on huggingface.co, which means that we store the files on S3/Cloudfront in a content-addressable way (i.e., the file

```
> name is its hash). Using content-addressable filenames means cache can't
> ever be stale.
>
> In terms of client-side caching from this library, we base our caching
> on the objects' entity tag (`ETag`), which is an identifier of a
> specific version of a resource [1]_. An object's ETag is: its git-sha1
> if stored in git, or its sha256 if stored in git-lfs.
```

References:

- [1] <https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/ETag>

****Parameters:****

`repo_id`(`str`)` : A namespace (user or an organization) name and a repo name separated by a `/``.`

`filename`(`str`)` : The name of the file in the repo.

`subfolder`(`str`, *optional*)` : An optional value corresponding to a folder inside the repo.

`repo_type`(`str`, *optional*)` : Set to `"dataset"`, `"space"` or `"kernel"` if downloading from a dataset, space or kernel repo, `None`` or `"model"` if downloading from a model. Default is `None``.

`revision`(`str`, *optional*)` : An optional Git revision id which can be a branch name, a tag, or a commit hash.

`endpoint`(`str`, *optional*)` : The Hub endpoint to send the request to. Defaults to the value of `HF_ENDPOINT``.

Download a snapshot of the repo[[`huggingface_hub.snapshot_download`]]

```
#### huggingface_hub.snapshot_download[[huggingface_hub.snapshot_download]]
```

[Source](https://github.com/huggingface/huggingface_hub/blob/v1.20.1/src/huggingface_hub/_snapshot_download.py#L106)

Download repo files.

Download a whole snapshot of a repo's files at the specified revision. This is useful when you want all files from a repo because you don't know which ones you will need `_a priori_`. All files are nested in a folder to keep their path and filename relative to that folder. You can also filter which files to download by using ``allow_patterns`` and ``ignore_patterns``.

If ``local_dir`` is provided, the file structure from the repo will be replicated in this location. When using this option, the ``cache_dir`` will not be used, and a ``.cache/huggingface/`` folder will be created at the root of ``local_dir`` to store some metadata related to the downloaded files. While this mechanism is not as robust as the main cache system, it's optimized for regularly pulling the latest version of a repository.

An alternative would be to clone the repo, but this requires git and git-lfs to be installed and properly configured. It is also not possible to filter which files to download when cloning a repository using git.

****Parameters:****

`repo_id`(`str`)` : A user or an organization name and a repo name separated by a `/``.`

`repo_type` (`str`, *optional*) : Set to "dataset", "space" or "kernel" if downloading from a dataset, space or kernel repo, None or "model" if downloading from a model. Default is None.`

`revision` (`str`, *optional*) : An optional Git revision id, which can be a branch name, a tag, or a commit hash.`

`cache_dir` (`str`, `Path`, *optional*) : Path to the folder where cached files are stored.`

`local_dir` (`str` or `Path`, *optional*) : If provided, the downloaded files will be placed under this directory.`

`library_name` (`str`, *optional*) : The name of the library to which the object corresponds.`

`library_version` (`str`, *optional*) : The version of the library.`

`user_agent` (`str`, `dict`, *optional*) : The user-agent info in the form of a dictionary or a string.`

`etag_timeout` (`float`, *optional*, defaults to 10) : When fetching ETag, how many seconds to wait for the server to send data before giving up, which is passed to httpx.request.`

`force_download` (`bool`, *optional*, defaults to False) : Whether the file should be downloaded even if it already exists in the local cache.`

`token` (`str`, `bool`, *optional*) : A token to be used for the download. - If True, the token is read from the HuggingFace config folder. - If a string, it's used as the authentication token.`

`headers` (`dict`, *optional*) : Additional headers to include in the request. Those headers take precedence over the others.`

`endpoint` (`str`, *optional*) : The Hub endpoint to send the request to. Defaults to the value of HF_ENDPOINT.`

`local_files_only` (`bool`, *optional*, defaults to False) : If True, do not download any files even if they are not in cache_dir or local_dir.`

`allow_patterns` (`list[str]` or `str`, *optional*) : If provided, only files matching at least one pattern are downloaded.`

`ignore_patterns` (`list[str]` or `str`, *optional*) : If provided, files matching any of the patterns are not downloaded.`

`max_workers` (`int`, *optional*) : Number of concurrent threads to download files (1 thread = 1 file download). Defaults to 8.`

`tqdm_class` (`tqdm`, *optional*) : If provided, overwrites the default behavior for the progress bar. Passed argument must inherit from tqdm.auto.tqdm or at least mimic its behavior. Note that the tqdm_class is not passed to each individual download. Defaults to the custom HF progress bar that can be disabled by setting HF_HUB_DISABLE_PROGRESS_BARS environment variable.`

`dry_run` (`bool`, *optional*, defaults to False) : If True, perform a dry run without actually downloading the files. Returns a list of [DryRunFileInfo](/docs/huggingface_hub/v1.20.1/en/package_reference/hf_api#huggingface_hub.DryRunFileInfo) objects containing information about what would be downloaded.`

****Returns:****

``str` or list of [DryRunFileInfo](/docs/huggingface_hub/v1.20.1/en/package_reference/hf_api#huggingface_hub.DryRunFileInfo)`

- If `dry_run=False`: Local snapshot path.
- If `dry_run=True`: A list of `[DryRunFileInfo](/docs/huggingface_hub/v1.20.1/en/package_referen`

ce/hf_api#huggingface_hub.DryRunFileInfo) objects containing download information.

Get metadata about a file

`get_hf_file_metadata[[huggingface_hub.get_hf_file_metadata]]`

`huggingface_hub.get_hf_file_metadata[[huggingface_hub.get_hf_file_metadata]]`

[Source](https://github.com/huggingface/huggingface_hub/blob/v1.20.1/src/huggingface_hub/file_download.py#L1560)

Fetch metadata of a file versioned on the Hub for a given url.

****Parameters:****

`url` (``str``) : File url, for example returned by `[hf_hub_url()](/docs/huggingface_hub/v1.20.1/en/package_reference/file_download#huggingface_hub.hf_hub_url)`.

`token` (``str`` or ``bool``, **optional**) : A token to be used for the download. - If ``True``, the token is read from the HuggingFace config folder. - If ``False`` or ``None``, no token is provided. - If a string, it's used as the authentication token.

`timeout` (``float``, **optional**, defaults to 10) : How many seconds to wait for the server to send metadata before giving up.

`library_name` (``str``, **optional**) : The name of the library to which the object corresponds.

`library_version` (``str``, **optional**) : The version of the library.

`user_agent` (``dict``, ``str``, **optional**) : The user-agent info in the form of a dictionary or a string.

`headers` (``dict``, **optional**) : Additional headers to be sent with the request.

`endpoint` (``str``, **optional**) : Endpoint of the Hub. Defaults to `.`

`retry_on_errors` (``bool``, **optional**, defaults to ``False``) : Whether to retry on errors (429, 5xx, timeout, network errors). If False, no retry for fast fallback to local cache.

****Returns:****

A `[HfFileMetadata](/docs/huggingface_hub/v1.20.1/en/package_reference/file_download#huggingface_hub.HfFileMetadata)` object containing metadata such as location, etag, size and `commit_hash`.

`HfFileMetadata[[huggingface_hub.HfFileMetadata]]`

`huggingface_hub.HfFileMetadata[[huggingface_hub.HfFileMetadata]]`

[Source](https://github.com/huggingface/huggingface_hub/blob/v1.20.1/src/huggingface_hub/file_download.py#L143)

Data structure containing information about a file versioned on the Hub.

Returned by `[get_hf_file_metadata()](/docs/huggingface_hub/v1.20.1/en/package_reference/file_download#huggingface_hub.get_hf_file_metadata)` based on a URL.

****Parameters:****

`commit_hash` (``str``, **optional**) : The `commit_hash` related to the file.

`etag` (``str``, **optional**) : Etag of the file on the server.

`location` (``str``) : Location where to download the file. Can be a Hub url or not (CDN).

size (`size`) : Size of the file. In case of an LFS file, contains the size of the actual LFS file, not the pointer.

xet_file_data (`XetFileData`, *optional*) : Xet information for the file. This is only set if the file is stored using Xet storage.

Caching

The methods displayed above are designed to work with a caching system that prevents re-downloading files. The caching system was updated in v0.8.0 to become the central cache-system shared across libraries that depend on the Hub.

Read the [cache-system guide](../guides/manage-cache) for a detailed presentation of caching at HF.

30. user (2026-06-21T14:03:24.975Z)

Web search results for query: "BtbN FFmpeg-Builds GPL LGPL difference libx264 libx265 GPLv3 ffprobe included"

Links: [{"title": "GitHub - BtbN/FFmpeg-Builds · GitHub", "url": "https://github.com/BtbN/FFmpeg-Builds"}, {"title": "Releases · BtbN/FFmpeg-Builds", "url": "https://github.com/BtbN/FFmpeg-Builds/releases"}, {"title": "BtbN/FFmpeg-Builds | DeepWiki", "url": "https://deepwiki.com/BtbN/FFmpeg-Builds"}, {"title": "Builds - CODEX FFmpeg @ gyan.dev", "url": "https://www.gyan.dev/ffmpeg/builds/"}, {"title": "Ragerworks", "url": "https://winget.ragerworks.com/package/BtbN.FFmpeg.LGPL.8.0"}, {"title": "Improve README Organization and Clarity · Issue #447 · BtbN/FFmpeg-Builds", "url": "https://github.com/BtbN/FFmpeg-Builds/issues/447"}, {"title": "FFmpeg 8.1.1 Download for Latest Version Windows - Free Builds by GyanD & BtbN", "url": "https://www.free-codecs.com/download/ffmpeg.htm"}, {"title": "FFmpeg (GPL shared variant, 7.1 release branch) BtbN", "url": "https://winget.ragerworks.com/package/BtbN.FFmpeg.GPL.Shared.7.1"}, {"title": "Build Variants and Targets | BtbN/FFmpeg-Builds | DeepWiki", "url": "https://deepwiki.com/BtbN/FFmpeg-Builds/4.1-build-variants-and-targets"}]

Based on the search results, here's information about BtbN FFmpeg-Builds and the GPL/LGPL difference:

GPL vs LGPL Variants

The LGPL variant lacks libraries that are GPL-only, most prominently libx264 and libx265. The LGPL variant omits GPL-licensed components, making it suitable for embedding in commercial software that cannot adopt GPL terms.

Build Options

The builds come in multiple variants: lgpl (lacking GPL-only libraries), gpl-shared (same as gpl but with shared libs instead of pure static executables), lgpl-shared (with lgpl dependencies), and nonfree-shared (with nonfree dependencies).

Licensing Configuration

The GPL variant enables `--enable-gpl`, while the LGPL variant omits it and uses `COPYING.LGPLV3`.

Regarding ffprobe, the search results indicate that GyanD Essentials contains ffmpeg.exe, ffplay.exe, and ffprobe.exe, though the specific inclusion in BtbN builds wasn't explicitly detailed in the search results. The build variants focus primarily on the licensing of encoding libraries like x264 and x265, which are the key difference between GPL and LGPL builds.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

31. user (2026-06-21T14:03:25.722Z)

Web search results for query: "gyan.dev ffmpeg builds GPLv3 license libx264 libx265 ffprobe winget Gyan.FFMpeg"

Links: [{"title": "Builds - CODEX FFMPEG @ gyan.dev", "url": "https://www.gyan.dev/ffmpeg/builds/"}, {"title": "FFmpeg Windows builds at gyan.dev - Page 2 - VideoHelp Forum", "url": "https://forum.videohelp.com/threads/399117-FFmpeg-Windows-builds-at-gyan-dev/page2"}, {"title": "GitHub - GyanD/codexffmpeg: Support for https://www.gyan.dev/ffmpeg . GitHub", "url": "https://github.com/GyanD/codexffmpeg"}, {"title": "Home - CODEX FFMPEG @ gyan.dev", "url": "https://www.gyan.dev/ffmpeg/"}, {"title": "FFmpeg Windows builds at gyan.dev - VideoHelp Forum", "url": "https://forum.videohelp.com/threads/399117-FFmpeg-Windows-builds-at-gyan-dev"}, {"title": "Resources - CODEX FFMPEG @ gyan.dev", "url": "https://www.gyan.dev/ffmpeg/resources/"}, {"title": "How to use FFmpeg: Installation, commands & examples ? Shotstack", "url": "https://shotstack.io/learn/how-to-use-ffmpeg/"}, {"title": "Releases - GyanD/codexffmpeg", "url": "https://github.com/GyanD/codexffmpeg/releases"}]

Based on the search results, here's what I found about gyan.dev FFmpeg builds:

Overview

The page hosts packages containing binaries of ffmpeg, ffprobe and ffplay. Builds require Windows 10 or later but may work on Windows 7/8 if UCRT is installed.

Licensing

All builds are 64-bit, static and licensed as GPLv3; the release full variant is also available as a shared build with development files. Supplementary tools are updated infrequently and are licensed as LGPLv3.

Build Variants and Libraries

The site offers builds in a couple of variants: the essentials build variant contains commonly used libraries, whereas the full build variant also contains most of the remainder. The essentials build includes libraries like libx264, libx265, and other commonly used codecs and utilities.

Installation via Package Managers

The following builds are available through package managers: release essentials via winget with "FFmpeg (Essentials Build)", release full via winget with ffmpeg, and release full shared via winget with "FFmpeg (Shared)".

Support Repository

The Releases section of the GitHub repository acts as a mirror for builds hosted at <https://www.gyan.dev/ffmpeg/builds/>

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

32. assistant (2026-06-21T14:03:31.779Z)

All facts now verified. Let me do a couple of final precise checks: the exact line references in the codebase files (the claims cite specific line numbers), and verify the `\_BUNDLED\_FONT` is RobotoSlab Apache-2.0 (the claim says libfreetype/drawtext at lines 179-185). Let me verify the line-number specifics against the actual file contents I already have, and confirm the resampling on resumable downloads for hf (the claim says "resumable downloads").

I already have enough from the files. Let me do final verifications on two items: huggingface_hub resumable downloads, and confirm whether `fetch\_to\_local` truly needs google-cloud-storage (relevant to the recommendation, not a claim per se).

33. user (2026-06-21T14:03:41.055Z)

Web search results for query: "huggingface_hub hf_hub_download resumable download HF_HUB_OFFLINE

offline mode"

```
Links: [{"title": "Working offline automatically after model has downloaded · Issue #6168 · huggingface/diffusers", "url": "https://github.com/huggingface/diffusers/issues/6168"}, {"title": "Utilities · Hugging Face", "url": "https://huggingface.co/docs/huggingface_hub/en/package_reference/utilities"}, {"title": "File Download System | huggingface/huggingface_hub | DeepWiki", "url": "https://deepwiki.com/huggingface/huggingface_hub/2.2-file-download-and-upload"}, {"title": "Download files from the Hub · Hugging Face", "url": "https://huggingface.co/docs/huggingface_hub/en/guides/download"}, {"title": "Ultimate guide to huggingface_hub library in Python", "url": "https://deepnote.com/blog/ultimate-guide-to-huggingfacehub-library-in-python"}, {"title": "Environment variables · Hugging Face", "url": "https://huggingface.co/docs/huggingface_hub/en/package_reference/environment_variables"}, {"title": "Installation · Hugging Face", "url": "https://huggingface.co/docs/transformers/main/en/installation"}, {"title": "huggingface_hub/src/huggingface_hub/file_download.py at main · huggingface/huggingface_hub", "url": "https://github.com/huggingface/huggingface_hub/blob/main/src/huggingface_hub/file_download.py"}]
```

Based on the search results, here's information about `huggingface_hub`, `hf_hub_download`, resumable downloads, and the `HF_HUB_OFFLINE` offline mode:

hf_hub_download Function

The `hf_hub_download()` function is the main function for downloading files from the Hub, which downloads the remote file, caches it on disk in a version-aware way, and returns its local file path.

Resumable Downloads

The `resume_download` parameter is deprecated without replacement, as `huggingface_hub` always resumes downloads whenever possible. The library is built to handle interruptions: if you cancel a download halfway, the partially downloaded file stays in cache with a temp name, and on the next call, it will resume where it left off rather than starting over.

HF_HUB_OFFLINE Environment Variable

You can programmatically check if offline mode is enabled using `is_offline_mode`, and offline mode is enabled by setting `HF_HUB_OFFLINE=1` as environment variable. Setting `HF_HUB_OFFLINE=1` will skip HTTP calls which speeds up your loading time.

The file download system supports automatic caching with content-addressable storage, offline mode operation, progress tracking, resumable downloads, and multiple storage backends.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

34. assistant (2026-06-21T14:03:50.796Z)

All facts verified. Let me confirm the exact line numbers in the codebase claims by checking the specific lines cited, since several claims hinge on precise `file:line` references (per the project's "lesson-verify-engine-surface-before-scoping" memory). I have the full files already. Let me cross-check the cited lines against what I read:

- `backend/utils/video.py` line 11: for `ffprobe`: line 11 is `duration_output = subprocess.check_output([`, line 12 is `"ffprobe", ...`. `ffmpeg` "line 33" is `cmd = [` (line 34 is `"ffmpeg", "-y",)`. Close but off by one.
- `compiler.py` line 113: `ffprobe`: line 113 IS `"ffprobe", "-v", "error",`. Correct.
- `compiler.py` line 253: `libx264`: line 253 IS `"-c:v", "libx264",`. Correct.
- `compiler.py` lines 179-185: `drawtext/overlay` watermark: lines 179-188 are the `drawtext/overlay`. Correct.
- `gcs.py` line 30: `fetch_to_local`: line 30 IS `def fetch_to_local(...)`. Correct.
- `native_wasb_runner.py` lines 80-97: `from_indexing_cfg` with `env` winning: lines 80-97 ARE that method, `env` wins via `env(...)` or `w.get(...)`. Correct.

- `doctor.py` lines 84-108 and 15-34`: lines 84-95 are `shutil`.which checks, 103 is `winget`
Gyan.FFmpeg, 116-124 GPU/torch, 26-34 cu124/cpu index. Correct.

Now I have everything needed. Let me compile the structured verdict.

35. user (2026-06-21T14:05:05.266Z)

Structured output provided successfully

36. assistant (2026-06-21T14:05:18.368Z)

I've completed the adversarial verification of all 12 claims. All claims were confirmed in substance ? none could be refuted ? but I found several minor corrections and one tension in the recommendation's mechanics that are worth folding in:

****Key corrections:****

- ****Units (Claim 1):**** PyPI limits are 100 MB / 10 GB (decimal), not 100 MiB / 10 GiB (binary). Conclusion unchanged.
- ****Citation weakness (Claim 3):**** `imageio-ffmpeg` issue #23 is a feature *request* to add `ffprobe`, not a statement that it isn't bundled. The fact holds; the citation is weaker than implied.
- ****Line drift:**** ``backend/utils/video.py`` invokes ``ffprobe`` at line 12 (not 11) and ``ffmpeg`` at line 34 (not 33). All ``compiler.py`` and ``gcs.py`` line refs are exact.
- ****Missed site (Claim 11):**** there is a ****second**** ``-c:v libx264`` at ``backend/utils/video.py`` line 42 (``extract_video_chunk``), beyond ``compiler.py`` line 253. Any LGPL/encoder-swap (FFMPEG-C) and the single-resolver helper must cover both `libx264` sites and all four subprocess call sites.
- ****Provenance gate (Claim 9) is stronger than stated:**** ``MODEL_ZOO.md`` mixes the authors' own MIT-covered WASB weights with third-party model weights (MonoTrack, TrackNetV2, BallSeg, etc.) on Google Drive. MIT on the repo code does not blanket-cover every weight blob ? mirror only the authors' own WASB badminton weights (MonoTrack is already flagged as a hazard in project memory).

****Tension in the recommendation mechanics (not a claim, but load-bearing):**** it says the public-bucket fetcher "reuses ``GCSStorage.fetch_to_local``" *and* "needs no google-cloud-storage dependency or credentials." Those conflict ? ``fetch_to_local`` (``backend/storage/gcs.py`` line 24) lazily imports ``google.cloud.storage`` and uses ADC. A credential-free public-bucket download is a *different* path (plain ``requests`/`urllib`` against ``https://storage.googleapis.com/<bucket>/<obj>``). The ``WASB_WEIGHTS`` env seam is reused either way, but the phrasing should be "add a small public-HTTPS fetcher (no google-cloud-storage dep) that writes the cache dir and sets ``WASB_WEIGHTS``."

The overall recommendation (split weights vs `ffmpeg`, don't vendor weights, don't redistribute `ffmpeg`, route through one resolver, extend ``doctor.py``) is sound and well-grounded.

Relevant files: ``C:\Users\avidu\Projects\badminton-highlight-indexer\backend\utils\video.py``,
``C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\stitcher\compiler.py``,
``C:\Users\avidu\Projects\badminton-highlight-indexer\backend\storage\gcs.py``,
``C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\detectors\native_wasb_runner.py``, ``C:\Users\avidu\Projects\badminton-highlight-indexer\scripts\doctor.py``.